ДИСЦИПЛИНА	Фронтенд и бэкенд разработка
ИНСТИТУТ	ИПТИП
КАФЕДРА	Индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Методические указания к практическим занятиям по дисциплине
ПРЕПОДАВАТЕЛЬ	Загородних Николай Анатольевич Краснослободцева Дарья Борисовна
СЕМЕСТР	4 семестр, 2025/2026 уч. год

Практическое занятие 20

Работа с NoSQL СУБД на примере MongoDB

Рассмотрим процесс работы с NoSQL системами управления базами данных на примере СУБД MongoDB. Решение практического задания осуществляется внутри соответствующей рабочей тетради, расположенной в СДО.

Введение в NoSQL и MongoDB

NoSQL (Not Only SQL) — это класс систем управления базами данных, отличающихся от традиционных реляционных СУБД отсутствием жесткой схемы данных и возможностью горизонтального масштабирования. В отличие от SQL-баз, где данные хранятся в таблицах с фиксированными столбцами, NoSQL использует документно-ориентированные, ключ-значение, графовые или колоночные модели.

MongoDB — одна из самых популярных документоориентированных NoSQL-систем, где данные хранятся в виде BSON-документов (бинарный JSON). Это позволяет гибко изменять структуру данных без необходимости изменения схемы, что особенно полезно в agile-разработке и при работе с неструктурированной информацией.

Установка и настройка MongoDB

Перед началом работы необходимо установить MongoDB. В рамках практического занятия будет рассмотрен процесс установки только для операционной системы Linux (Ubuntu). Допустимо использовать любой другой удобный дистрибутив на виртуальной машине или в качестве операционной системы устройства.

Процесс установки

Обновим пакеты и установим утилиту `curl`

```
sudo apt update
sudo apt install -y curl
```

Импортируем открытый ключ для MongoDB в систему

```
curl -fsSL https://pgp.mongodb.com/server-7.0.asc | sudo gpg -o
/usr/share/keyrings/mongodb-server-7.0.gpg --dearmor
```

Добавим репозиторий MongoDB 7.0 в директорию `/etc/apt/sources.list.d`

```
echo "deb [ arch=amd64,arm64 signed-by=/usr/share/keyrings/mongodb-server-
7.0.gpg ] https://repo.mongodb.org/apt/ubuntu jammy/mongodb-org/7.0
multiverse" | sudo tee /etc/apt/sources.list.d/mongodb-org-7.0.list
```

Обновим список репозиториев и установим пакет MongoDB

```
sudo apt update
sudo apt install -y mongodb-org
```

Запустим службу и проверим статус её работы

```
sudo systemctl start mongod
sudo systemctl status mongod
```

Процесс настройки

Перейдем в командную оболочку

```
mongosh
```

Добавим административного пользователя

```
use admin
```

```
db.createUser( { user: "YourMongoAdmin", pwd: "1234", roles: [ { role:
"userAdminAnyDatabase", db: "admin" }, "readWriteAnyDatabase" ] } )
```

Включение аутентификации

Чтобы запустить пользовательскую аутентификацию, необходимо внести изменения в конфигурационный файл MongoDB `mongod.conf`. Пока вы не сделаете это и не

перезапустите службу MongoDB, пользователи будут подключаться к вашим базам данным без аутентификации. Однако, при этом они не смогут просматривать и редактировать данные пока не предоставят корректные имя пользователя и пароль.

Откройте для редактирования конфигурационный файл `mongod.conf`, например, при помощи текстового редактора `nano`:

```
sudo nano /etc/mongod.conf
```

В этом файле найдите строку `#security` и раскомментируйте её (удалите в начале строки символ `#`).

Следующей строкой добавьте текст:

```
authorization: enabled
```

Причём, необходимо убедиться, что в строке `security` нет никаких пробелов в её начале. Строка же `authorization: enabled`, напротив, имеет два пробела отступа в начале строки. Отредактированный текст должен выглядеть следующим образом:

```
# how the process runs
processManagement:
  timeZoneInfo: /usr/share/zoneinfo

security:
  authorization: enabled

#operationProfiling:

#replication:
```

Подключение MongoDB к Node.js-приложению

Для работы с MongoDB в Node.js используется библиотека `mongoose` — ODM (Object Data Modeling), которая предоставляет удобный API для взаимодействия с базой.

1. **Инициализация проекта**
   Создаем новый проект и устанавливаем зависимости:
   ```
   npm init -y
   npm install mongoose express
   ```
2. **Подключение к базе данных**
   Создаем файл `server.js` и подключаем MongoDB:
   ```
   const mongoose = require('mongoose');
   const express = require('express');
   const app = express();
   ```

```
// Подключение к MongoDB
mongoose.connect('mongodb://YourMongoAdmin:1234@localhost:27017/admin',
{
    useNewUrlParser: true,
    useUnifiedTopology: true
})
.then(() => console.log('Connected to MongoDB'))
.catch(err => console.error('Connection error:', err));

app.use(express.json());

app.listen(3000, () => {
    console.log('Server is running on http://localhost:3000');
});
```

Определение модели данных

Mongoose использует **схемы** (Schemas) для описания структуры документов. На основе схем создаются **модели** (Models), которые обеспечивают взаимодействие с коллекциями в MongoDB.

Пример модели для сущности "Пользователь":

```
const userSchema = new mongoose.Schema({
    name: { type: String, required: true },
    email: { type: String, required: true, unique: true },
    age: { type: Number, min: 18 },
    createdAt: { type: Date, default: Date.now }
});

const User = mongoose.model('User', userSchema);
```

Здесь:

- `required: true` — поле обязательно для заполнения.
- `unique: true` — значение должно быть уникальным в коллекции.
- `default` — значение по умолчанию.

CRUD-операции в MongoDB

1. Создание документа (Create)

Добавление нового пользователя:

```
app.post('/users', async (req, res) => {
    try {
```

```
        const user = new User(req.body);
        await user.save();
        res.status(201).send(user);
    } catch (err) {
        res.status(400).send(err.message);
    }
});
```

Пример HTTP-запроса (Postman или Fetch):

```
POST /users
Content-Type: application/json

{
    "name": "Михаил",
    "email": "mikhail@alekhin.ru",
    "age": 20
}
```

2. Чтение данных (Read)

Получение всех пользователей:

```
app.get('/users', async (req, res) => {
    try {
        const users = await User.find();
        res.send(users);
    } catch (err) {
        res.status(500).send(err.message);
    }
});
```

Поиск по условию (например, пользователи старше 18 лет):

```
const users = await User.find({ age: { $gt: 18 } });
```

3. Обновление данных (Update)

Изменение данных пользователя:

```
app.patch('/users/:id', async (req, res) => {
    try {
        const user = await User.findByIdAndUpdate(
            req.params.id,
            req.body,
            { new: true } // Возвращает обновленный документ
```

```
        );
        if (!user) return res.status(404).send('User not found');
        res.send(user);
    } catch (err) {
        res.status(400).send(err.message);
    }
});
```

4. Удаление данных (Delete)

Удаление пользователя:

```
app.delete('/users/:id', async (req, res) => {
    try {
        const user = await User.findByIdAndDelete(req.params.id);
        if (!user) return res.status(404).send('User not found');
        res.send(user);
    } catch (err) {
        res.status(500).send(err.message);
    }
});
```

Интеграция с фронтендом

Для демонстрации взаимодействия с фронтендом можно использовать Fetch API:

```
// Получение списка пользователей
fetch('http://localhost:3000/users')
    .then(response => response.json())
    .then(data => console.log(data));
```

Оптимизация и дополнительные возможности

- **Индексы** — ускоряют поиск по часто используемым полям:

  ```
  userSchema.index({ email: 1 }); // Создание индекса
  ```

- **Агрегации** — сложные запросы с группировкой:

  ```
  const stats = await User.aggregate([
      { $group: { _id: null, averageAge: { $avg: "$age" } } }
  ]);
  ```

Практическое задание

Создайте простое API для управления списком пользователей. Необходимо реализовать серверное приложение с подключением базы данных MongoDB (можно

использовать любую не реляционную, документоориентированную базу данных) для хранения информации о пользователях.

Сущность `Пользователь` должна включать следующий минимальный набор полей (можно добавлять свои):

Поле	Тип данных	Описание
ID	Integer	Уникальный идентификатор пользователя
first_name	Varchar	Имя пользователя
last_name	Varchar	Фамилия пользователя
age	Integer	Возраст пользователя
created_at	Timestamp	Время создания пользователя в unix
updated_at	Timestamp	Время обновления пользователя в unix

Список маршрутов должен включать следующий минимальный перечень энд-поинтов (можно добавлять свои):

Адрес	Метод	Описание
/api/users	POST	Создание нового пользователя
/api/users	GET	Получение списка пользователей
/api/users/:id	GET	Получение конкретного пользователя
/api/users/:id	PATCH	Обновление информации пользователя
/api/users/:id	DELETE	Удаление пользователя

> ⚠ **Warning**
>
> Обязательно связать логику каждого маршрута с No-SQL базой данных!

Формат отчета

В качестве ответа на задание необходимо прикрепить ссылку на репозиторий с реализованной практикой.